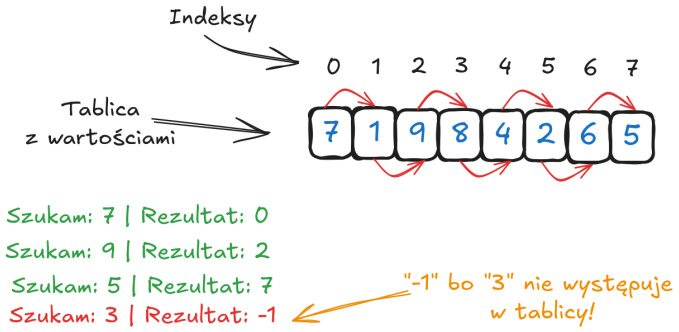
1 Wprowadzenie

Praca „na danych” oznacza konieczność wykorzystywania danych, co może przykładowo przejawiać się w ich: przenoszeniu z jednego punktu do drugiego, powielaniu, modyfikowaniu, filtrowaniu, porządkowaniu, przeszukiwaniu.

Formą porządkowania danych może być ich posortowanie, a to już umiemy zrobić (po wykonaniu pierwszego ćwiczenia). Wiemy również, że problem sortowania nie ma jednego rozwiązania, a ich mnogość zależy od wielu czynników (rodzaju danych, wielkości porządkowanej kolekcji, dostępności zasobów (zarówno sprzętowych jak i ludzkich) oraz bibliotek).

Powinniśmy już mieć również świadomość w jaki sposób możemy przeszukiwać kolekcję pod kątem występowania w niej określonego elementu. W przypadku nieuporządkowanej (nieposortowanej) kolekcji jesteśmy ograniczeni przez podejście naiwne (liniowe; sekwencyjne), czyli stosując **algorytm wyszukiwania sekwencyjnego** (rys. 1) przechodzimy od początku do końca kolekcji sprawdzając element po elemencie dopasowanie do wyszukiwanej wartości. Niewątpliwą zaletą tego podejścia jest to, że działa ono dla nieuporządkowanego zestawu wartości. Wadą natomiast jest złożoność takiego przeszukiwania, które charakteryzuje się złożonością liniową ($O(n)$).



Rysunek 1: Wizualizacja algorytmu przeszukiwania liniowego. Mamy nieuporządkowaną kolekcję ośmiu elementów, którą przechodzimy od lewej (indeks 0) do prawej (indeks 7). Jeśli poszukiwany element znajduje się w tablicy, to zwracamy indeks, pod którym występuje. W przeciwnym razie zwracamy (umownie): -1.

Alternatywnym pomysłem, gdy mamy uporządkowaną (posortowaną) kolekcję, jest użycie **algorytmu wyszukiwania binarnego** opierającego się na podejściu *dziel i zwyciężaj*. Pierwsze spotkanie z tym problemem mieliśmy w trakcie zajęć wprowadzających, dlatego nie będziemy teraz ponownie szczegółowo analizowali tego problemu. Trzeba jednak odnotować, że uzyskiwane rezultaty działania tego algorytmu powinny być identyczne jak w przypadku naiwnym, czyli zwracamy indeks tablicy, pod którym występuje poszukiwana wartość, lub `-1`, jeśli wartość nie występuje w kolekcji.

Złożoność wyszukiwania (binarnego) elementu pośród n-elementowej tablicy jest określana jako $O(\log(n))$, a wynika to z tego, że dla przykładowego $n = 100$ zwrócenie wyniku powinno nastąpić po maksymalnie $\lfloor log_2(100) + 1 \rfloor = 7$ iteracjach pętli porównującej wyszukiwaną wartość i poławiającej (dzielącej na „połowę") przeszukiwany zakres.

Czy można to sobie jakoś lepiej zwizualizować? Można! Proszę narysować zrównoważone drzewo poszukiwań binarnych reprezentujące naszą uporządkowaną kolekcję, a następnie proszę sprawdzić, ile porównań trzeba wykonać, żeby dojść do elementu znajdującego się na najniższym poziomie w drzewie.

Bezdyskusyjnie algorytm wyszukiwania binarnego (dla uporządkowanej kolekcji) bije na głowę algorytm liniowy. Jest to tym bardziej zauważalne, im większa jest liczba elementów w kolekcji. A teorię $(O(\log(n) < O(n))$ można bez problemu potwierdzić praktycznym przykładem.

Pojawia się jednak dodatkowe pytanie: **czy można szybciej**? Czy można uniezależnić czas wyszukiwania elementu od wielkości kolekcji?

Odważmy się i postawmy to pytanie! Czy można osiągnąć stałą złożoność ($O(1)$)? I teraz, od razu, idąc na skróty, odpowiedzmy: można!

1.1 Adresowanie bezpośrednie

Można stworzyć kolekcję, która będzie miała stałą ($O(1)$) złożoność dla operacji wyszukiwania elementów. Ale dodatkowo ta kolekcja może mieć taką samą (stałą) złożoność dla operacji dodawania i usuwania elementów. Brzmi zaskakująco? To spróbujmy rzucić na ten tajemniczy problem nieco światła.

Jednym z pomysłów na to zagadnienie jest wykorzystanie **adresowania bezpośredniego**. Wyobraźmy sobie, że każdy element, który chcemy przechowywać, da się jednoznacznie odwzorować na unikalną liczbę całkowitą, którą na potrzeby tego rozważania będziemy nazywać kluczem. Jeżeli jesteśmy w stanie określić, że nasze klucze `key` są z zakresu $key \in < 0; m - 1 >$, a m nie jest zbyt dużą wartością, to możemy wszystkie nasze elementy umieścić w tablicy $T[0...m-1]$. Wyszukanie elementu będzie wymagało od nas jedynie wyliczenia wartości klucza `key` i sprawdzenia, czy w tablicy T pod danym indeksem jest wartość inna niż `null` (`T[key] == null`).

Wstawianie elementu będzie przeprowadzane poprzez zastąpienie w tablicy T pod indeksem key wartości `null` wstawianym elementem (czyli: `T[key] = element`). A usunięcie będzie wyglądało analogicznie (po wyliczeniu klucza dla wartości, której chcemy się pozbyć z kolekcji): `T[key] = null`.

Istotną wadą tego pomysłu jest założenie, że nasze klucze muszą być mniejsze od niezbyt dużego m ($key < m$). Dlaczego mamy takie ograniczenie? Ponieważ m określa nam rozmiar tablicy `T`, a w pamięci komputera nie możemy dysponować nieograniczoną pamięcią.

Dodatkową wadą jest fakt, że nawet jeśli naszych wartości nie będzie zbyt wiele, ale ich odwzorowanie na klucze będzie generowało duże wartości (czyli: $n = 100$, $max_key = 1_000_000$), to stosując podejście adresowania bezpośredniego zaalokujemy tablicę o rozmiarze 1_000_000, a 999_900 jej komórek

będzie wypełnionych wartością *null* (99.99% komórek tabeli będzie niewykorzystanych).

Jak widzimy, adresowanie bezpośrednie jest stosunkowo prostym zagadnieniem, ale nadaje się bardziej do rozważań teoretycznych niż praktycznych zastosowań w codziennych, typowych (programistycznych) zadaniach.

1.2 Tablica z haszowaniem

Wymienione wady adresowania bezpośredniego nie zachęcają do wykorzystywania go w praktycznych zastosowaniach. Nie przekreślają one jednak samej idei, którą wystarczy trochę zmodyfikować, aby uzyskać rozwiązanie znacznie bardziej nadające się do użytku nie tylko na polu akademickim ale również w świecie komercyjnym.

Tym „ulepszeniem" jest **tablica z haszowaniem** (tablica mieszająca). Zobaczyliśmy, że w adresowaniu bezpośrednim element zmapowany na klucz `key` jest umieszczony w tablicy `T` pod indeksem `key` (`T[key] = element`). A w przypadku tablicy z haszowaniem T_H nasz element trafia pod indeks wyliczony przez **funkcję haszującą** (funkcję mieszającą) $h(key)$, czyli `T[h(key)] = element`.

Ideą tej funkcji $h(key)$ jest odwzorowanie klucza `key` na indeks tablicy $T_H[0...m-1]$, gdzie m jest rozmiarem tablicy z haszowaniem, który zazwyczaj jest znacznie mniejszy od `max_key` (czyli największej wartości z uniwersum wszystkich kluczy). Wypada zauważyć, że wartość wyliczona przez funkcję $h(key)$ jest nazywana **wartością haszującą** klucza key.

Jeśli w ten sposób redukujemy rozmiar tablicy T_H (w porównaniu do adresowania bezpośredniego), to tym samym zmniejszamy liczbę pustych, nieużywanych indeksów tablicy (wypełnionych przez `null`).

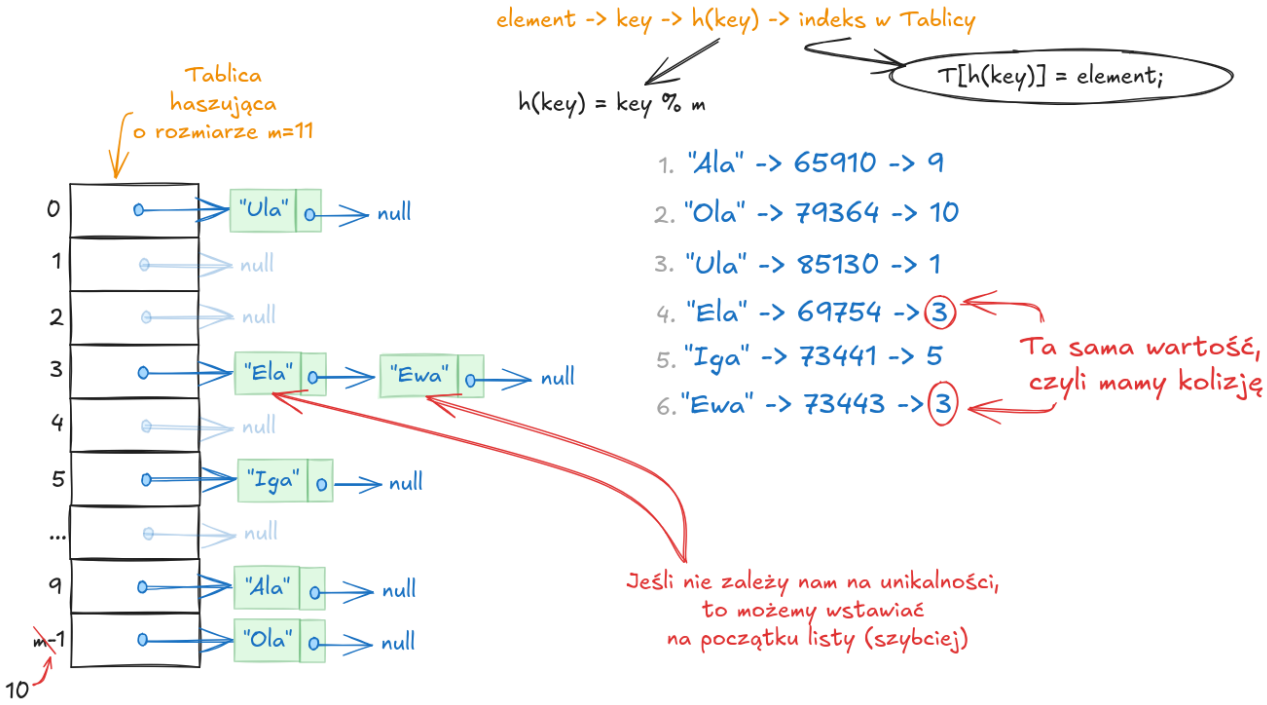
Wobec powyższego powstaje oczekiwanie, że funkcja haszująca $h(key)$ będzie dobrze rozpraszała wartości po tablicy unikając powtórzeń wyliczonych indeksów.

Należy sobie teraz postawić pytanie: co zrobimy, jeżeli to oczekiwanie nie zostanie spełnione? Co zrobimy, jeśli dla dwóch różnych elementów zostaną wyliczone dwa różne unikalne klucze key_1, key_2, ale dla tych dwóch różnych kluczy funkcja haszująca $h(key)$ zwróci taką samą wartość haszującą ($key_1 \neq key_2, h(key_1) = h(key_2)$)?

Taką sytuację – sytuację, gdy dla dwóch różnych kluczy wyliczamy takie same wartości haszujące – nazywamy **kolizją**. Podobnie jak w życiu tak i w programowaniu wolimy (a przynajmniej powinniśmy woleć, jeśli nie mamy skłonności masochistycznych) unikać kolizji. Niestety te kolizje czasami występują, dlatego warto ustalić, co możemy zrobić w takiej sytuacji.

Jednym z pomysłów (i na tym koncentrujemy się w tym ćwiczeniu) jest **rozwiązanie kolizji metodą łańcuchową** (rys. 2), co oznacza, że w tablicy mieszającej T_H pod indeksem wyliczonym przez funkcję haszującą jest lista liniowa, którą wypełniamy elementami o tej samej wartości haszującej.

Dla tablicy haszującej możemy wprowadzić **współczynnik zapełnienia** α



Rysunek 2: Przykład tablicy mieszającej (o rozmiarze m=11) z rozwiązywaniem kolizji metodą łańcuchową. Doszło do kolizji dla elementów "Ela" oraz "Ewa" – obydwa otrzymały taką samą wartość z funkcji haszującej. Wybraną funkcją haszującą było haszowanie modularne.

tabeli T_H, taki że:

$$\alpha = \frac{n}{m}$$

, gdzie n jest liczbą elementów w tablicy, a m jest rozmiarem tablicy.

Przyjmuje się, że dla $\alpha < 0.75$ tablica mieszająca będzie działała sprawnie (złożoność $O(1)$ dla każdej z operacji: wyszukiwania, wstawiania, usuwania).

W praktyce (przy rozwiązywaniu kolizji metodą łańcuchową), nawet dla α nieco większego od jedności, oczekiwana złożoność jest osiągalna. Istotnym determinantem takiej skuteczności jest dobra funkcja haszująca, która będzie rozpraszała elementy po całej tablicy, a tym samym będzie unikała gromadzenia nadmiernej liczby elementów pod jednym indeksem. Skrajnym przypadkiem – dla bardzo źle (dyskwalifikująco źle!) działającej funkcji haszującej – byłoby umieszczenie wszystkich elementów pod jednym indeksem tablicy mieszającej. Dlaczego? Ponieważ wtedy działanie tablicy mieszającej do poszukiwania elementu byłoby utożsamiane z działaniem naiwnego algorytmu przeszukiwania liniowego, czyli zamiast złożoności $O(1)$ otrzymalibyśmy $O(n)$.

1.3 Funkcje mieszające

Wybór i implementacja funkcji haszującej (dla metody łańcuchowej) nie musi być zadaniem trudnym, pod warunkiem, że spełnione zostaną dodatkowe ograniczenia.

Wyróżniamy trzy warianty funkcji mieszających: haszowanie modularne, haszowanie przez mnożenie, haszowanie uniwersalne.

1.3.1 Haszowanie modularne

$$h(key) = key\%m$$

, gdzie:

- % – operacja modulo;
- m – rozmiar tablicy (zalecane jako liczby pierwsze niezbyt bliskie potęgom 2).

1.3.2 Haszowanie przez mnożenie

$$h(key) = \lfloor m \cdot ((key \cdot A)\%1) \rfloor$$

, gdzie:

- $(key \cdot A)\%1$– część ułamkowa z $(key \cdot A)$;
- A – stała wpływająca na rozrzut wartości (prof. D. Knuth zaleca $A = \frac{\sqrt{5}-1}{2}$);
- m – rozmiar tablicy (teoretycznie można wybrać dowolnie).

1.3.3 Haszowanie uniwersalne

Celem jest zredukowanie prawdopodobieństwa kolizji do minimum. Tworzymy rodzinę funkcji haszujących, z której losujemy jedną funkcję do wyliczenia wartości haszującej. Przykładowy pomysł jest zapisany jako wzór dostępny w materiałach w ISOD-zie.

$$h(key) = (\sum_{i=0}^{r} a_i \cdot x_i)\%m$$

, gdzie:

- m – rozmiar tablicy (liczba pierwsza);
- key – klucz składa się z $r+1$ bajtów, czyli $key =< x_0, ..., x_r >$, a poszczególne bajty są $x_i \leqslant m$;
- a_i – wylosowana liczba ze zbioru $a_i \in < 0, m-1 >$.

2 Zadania

2.1 Zadanie 0.5

Proszę uporządkować swoje repozytorium, aby organizacja projektów wyglądała jak poniżej. Oczywiście w tej strukturze mogą znajdować się również inne katalogi i pliki, ale proszę dokładnie odwzorować przedstawiony porządek projektów.

```
.korzeń_repozytorium
+-- CW_0
|   |
|   +-- pliki_i_katalogi_z_zajęć
|
+-- CW_1
|   +-- AiSD2026ZEx1
|       +-- pom.xml
|
+-- CW_2
    +-- AiSD2026ZEx2
        +-- pom.xml
```

2.2 Zadanie 1

Do tablicy mieszającej (z haszowaniem modularnym) wkładamy 100_000 elementów, a następnie, rozpoczynając mierzenie czasu, próbujemy wyszukać każdy z tych elementów. Po wyszukaniu wszystkich elementów zatrzymujemy pomiar czasu.

Proszę uzupełnić dwie tabele (1, 2), pokazujące, jak zmienia się czas wyszukiwania wszystkich elementów w zależności od rozmiaru tablicy mieszającej.

Lp.	Rozmiar	Zmierzony czas [???]	Wsp. zapełnienia α
1.	4096		
2.	8192		
3.	16384		
4.	32768		
5.	65536		
6.	131072		
7.	262144		
8.	524288		
9.	1048576		
10.	2097152		

Tabela 1: Tabela prezentuje czas wyszukania wszystkich elementów w tabeli mieszającej (z haszowaniem modularnym) o rozmiarze będącym potęgą liczby 2.

Lp.	Rozmiar	Zmierzony czas [???]	Wsp. zapełnienia α
1.	5101		
2.	8297		
3.	16493		
4.	32909		
5.	65651		
6.	131111		
7.	263023		
8.	524309		
9.	1048583		
10.	2097169		

Tabela 2: Tabela prezentuje czas wyszukania wszystkich elementów w tabeli mieszającej (z haszowaniem modularnym) o rozmiarze będącym liczbą pierwszą.

Komentarz (a.k.a. wnioski): TODO ————————.

2.3 Zadanie 2

Proszę przygotować wykresy (histogramy) pokazującr, ile jest łańcuchów poszczególnych długości dla dwóch skrajnych rozmiarów tablic mieszających (5101, 2097169) w przypadku dwóch funkcji haszujących (modularnej, przez mnożenie).

Miejsce na wykresy (4 histogramy) proszę przygotować sobie samodzielnie.
Komentarz (a.k.a. wnioski): TODO ——————.

2.4 Zadanie 3

Proszę porównać czas wyszukania 100_000 elementów w zależności od: rozmiaru tablicy mieszającej, użytej funkcji mieszającej. Dla haszowania przez mnożenie proszę zaproponować samemu dwie wartości stałej A. Proszę skomentować uzyskane wyniki.

Lp.	Rozmiar	Zmierzony czas [???]				α
		H. modularne	H. przez mnożenie			
			$A = \frac{\sqrt{5}-1}{2}$)	A =??	A =??	
1.	5101					19.60
2.	8297					12.05
3.	16493					6.06
4.	32909					3.04
5.	65651					1.52
6.	131111					0.76
7.	263023					0.38

Tabela 3: Tabela pokazuje, jak zmienia się czas wyszukania 100_000 elementów w tablicy mieszającej w zależności od użytej funkcji mieszającej i rozmiaru tej tablicy.

Komentarz (a.k.a. wnioski): TODO ——————.

2.5 Zadanie 4*

Proszę, wykorzystując gotowe oraz własne testy jednostkowe, zweryfikować poprawność otrzymanego i uzupełnionego kodu.
Uwaga! To, co poniżej, tylko dla tych, którzy się nudzą.

Proszę zwrócić uwagę na niedoskonałość (wadę) metody testowej `should_CorrectlyAddThreeDifferentElems_WhenHashSizeIsOne()` w klasie testowej `GeneralHashListChainingTest`. Metoda ta nie nadaje się do ogólnej klasy testowej, ponieważ nie jest niezależna od konkretnej implementacji interfejsu `HashTable` (czyli narusza ostatnią zasadę SOLID). Proszę zaproponować zmianę.
Krótki opis proponowanej zmiany: TODO ——————.

3 Na zakończenie

To już prawie koniec, ale nie wolno nam zapomnieć, że:

- wszystkie zmiany (cały kod źródłowy) muszą trafić do repozytorium;
- ostatnioą zmianę (*commit*), przed końcem zajęć, należy oznaczyć tagiem ze znacznikiem czasowym **CW_2_EOT**. A ostatnią zmianę (*commit*) należy oznaczyć tagiem ze znacznikiem czasowym **CW_2_FINAL**;

- końcową wersja tego dokumentu (PDF) należy umiecić w ISOD-zie.